

Containers, Security, and Echo Chambers

Sunday, May 20, 2018 · 7 min read

TABLE OF CONTENTS

More than One Layer of Security Required

Breaking Changes

So why all the confusion and FUD?

Operability

There seems to be some confusion around sandboxing containers as of late, mostly because of the recent launch of [gvisor](#). Before I get into the body of this post I would like to make one thing clear. I have no problem with gvisor itself. I think it is very technically “cool.” I do have a problem with the messaging around it and marketing.

There is a large amount of ignorance towards the existing defaults to make containers secure. Which is crazy since I have written many blog posts on it and given many talks on the subject. But I digress, let’s focus on the part of the README that mentions sandboxing with SELinux, Seccomp, and Apparmor. It says: “However, in practice it can be extremely difficult (if not impossible) to reliably define a policy for arbitrary, previously unknown applications, making this approach challenging to apply universally.”

Greetings. Reporting for duty. Literally I am the person who can do that. I was the person who *did* do that. I added the default Seccomp profile to Docker and maintained the default Apparmor profile. I have also done A LOT of research with regard to Linux kernel isolation and making containers secure. I also literally reported for duty, two years ago and made the patch to add the Seccomp annotation to Kubernetes... with the hopes of eventually turning on a default filter.



Not found

[@nathanmccauley](#) [@brendandburns](#) [@kelseyhightower](#) [@thockin](#) I already offered to help

— jessie frazelle (@jessfraz) [April 5, 2016](#)

All big organizations have problems with “not invented here.” I tried my very best to inform everyone how these sandboxing mechanisms work but I am going to try one last time here.

More than One Layer of Security Required

In my last blog post, [Hard Multi-Tenancy in Kubernetes](#), I mentioned this as well. It is also a good read if you want to learn about the thought process for secure isolation. To be truly secure you need more than one layer of security so that when there is a vulnerability in one layer, the attacker also needs a vulnerability in another layer to bypass the isolation mechanism.

In Docker, we worked really hard to create secure defaults for the container isolation itself. I then tried to bring all those up the stack into orchestrators.

Container runtimes have security layers defined by Seccomp, Apparmor, kernel namespaces, cgroups, capabilities, and an unprivileged Linux user. All the layers don't perfectly overlap, but a few do.

Let's go over some of the ones that do overlap. I could do them all, but I would be here all day. The `mount` syscall is prevented by the default Apparmor profile, default Seccomp profile, and `CAP_SYS_ADMIN`. This is a neat example as it is literally three layers. Wow.

Everyone's favorite thing to complain about in containers or to prove that they know something is creating a fork bomb. Well this is actually easily preventable. With the PID cgroup you can set a max number of processes per container.

What about things that are not namespaced by the linux kernel..?

`CAP_SYS_TIME` prevents people from changing the time inside containers. And the default Seccomp profile prevents modifications or interacting with the kernel keyring.

If you would like a list of all the syscalls prevented by the default Seccomp profile, I behoove you to read the list [here](#). It also has descriptions of each.

Two years ago, there was [a great Whitepaper from NCC Group about hardening linux containers](#). Still to this day I get all the good feels when I see all the mentions of my work in it. But if you have any hesitations towards the defaults in Docker or otherwise I suggest you educate yourself first.

I will call out my favorite chart here though. Below shows the defaults from various container runtimes as of two years ago. Note the strong defaults in Docker. The paper also explains at length the defaults and would be a less biased version than me explaining myself.

Available Container Security Features, Requirements and Defaults			
Security Feature	LXC 2.0	Docker 1.11	CoreOS Rkt 1.3
User Namespaces	Default	Optional	Experimental
Root Capability Dropping	Weak Defaults	Strong Defaults	Weak Defaults
Procs and Sysfs Limits	Default	Default	Weak Defaults
Cgroup Defaults	Default	Default	Weak Defaults
Seccomp Filtering	Weak Defaults	Strong Defaults	Optional
Custom Seccomp Filters	Optional	Optional	Optional
Bridge Networking	Default	Default	Default
Hypervisor Isolation	Coming Soon	Coming Soon	Optional
MAC: AppArmor	Strong Defaults	Strong Defaults	Not Possible
MAC: SELinux	Optional	Optional	Optional
No New Privileges	Not Possible	Optional	Not Possible
Container Image Signing	Default	Strong Defaults	Default
Root Iteration Optional	True	False	Mostly False

The [non-events](#) are also an interesting read.

Breaking Changes

A lot of the push back I got from the default Seccomp profile was related to it being a breaking change.

I get that this is very scary. No really I get it. When we added it to Docker, guess who got paged when the Docker apt repo was down and it was on the front page of hacker news with tech bros crying: me. So I was absolutely

horrified at the thought of making a breaking change that might land on the front page of hacker news as well.

The last thing I ever wanted to do was cause a breaking change. That shit was terrifying. I lost sleep over weeks worrying about it. I tested every single Dockerfile on GitHub with the default profile. I ran `strace` on each for EPERMS and sent them all to elastic search. I made a project just for it: [strace2elastic](#). It's super dumb but was fun.

By the time we released I knew I had done at least everything in my power to make sure we didn't break anyone. The release actually went really well too. However, when you try to explain this to other projects they of course have their doubts, which I do not blame them for. I wish there was a better way to trust the genuine people who just want to help in open source.

So why all the confusion and FUD?

Well, it's simple really. Marketing. The tech never sells itself. It's all about marketing.

When you work at a large organization you are surrounded by an echo chamber. So if everyone in the org is saying "containers are not secure," you are bound to believe it and not research actual facts. To be clear I am not saying containers are secure, literally nothing is secure. Spreading FUD while ignorant or not doing proper research is harmful to the facts and hard work many people put in to making containers at least decently isolated by default.

Operability

There is another problem I have with gvisor. In my opinion, I think it would be quite hard to operate. People enjoy debugging with certain workflows and reinventing syscalls is going to be quite hard to debug. Just look up one of Bryan Cantrill's rants on unikernels which are harder to debug as well.

I believe it is putting a lot of extra burden on the operator to learn how to operate. At the end of the day you are left with a decision to trust or research the container security defaults or use a new runtime that re-implements all the syscalls in user-space and has poorer performance because of that. I also have yet to see a report on the fact that running in user-space is actually more secure. The implementation could be closely related to that of user mode linux and even user mode linux was never fully vetted for multi-tenancy so what are you really gaining. I truly believe it cannot be possibly more secure than the defaults for containers are today and surely it is not as secure as a real hypervisor. But, again, nothing is actually secure.

I am not trying to throw shade at gvisor but merely clear up some FUD in the world of open source marketing. I truly believe that people choosing projects to use should research into them and not just choose something shiny that came out of Big Corp. I also believe that people at Big Corp should embrace the work and ideas of people outside their echo chamber. Sometimes they even work in the echo chamber but just don't abide by the echo chamber beliefs.

Open your minds and hearts to the ideas of other people and you might just create something you never thought was possible in the first place.

Update: See [James Bottomley's research on Horizontal Attack Profile](#) which shows gVisor uses more syscalls than a standard docker container.

© Jessie Frazelle 2022

@jessfraz